

UNIT 2

Optimization Dynamics & Regularization

Prerequisites covered: Perceptron · Forward & Backward Propagation · Loss Functions · Activation Functions

Roadmap of This Unit

Once a neural network's architecture is fixed — perceptron units wired into layers, activation functions chosen, and a loss function defined — the remaining question is how to actually make the network learn well. Forward propagation produces a prediction, the loss function scores it, and backpropagation computes gradients. This unit is about everything that happens next: how those gradients are turned into weight updates (optimization), how training is kept numerically stable (normalization), and how the network is stopped from simply memorizing the training data (regularization).

The unit is organized into six practical blocks, each building on the last:

- Gradient Descent Techniques — Batch GD, Mini-Batch GD, SGD
- Advanced Optimizers — Momentum, Nesterov Accelerated Gradient (NAG)
- Adaptive Learning Methods — AdaGrad, RMSProp, Adam
- Gradient Challenges — Vanishing and Exploding Gradients
- Normalization Techniques — Batch Normalization, Layer Normalization
- Regularization & Tuning — L1/L2, Dropout, Early Stopping, Data Augmentation, Grid/Random Search, Bayesian Optimization

1. Gradient Descent Techniques

Gradient Descent is the core algorithm used to minimize the loss function

$L(w)$

with respect to the network's weights w .

$$w := w - \eta \cdot \nabla L(w)$$

Here η (eta) is the learning rate, a hyperparameter controlling the size of each step.

The basic update rule is:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

where:

- W = model parameter
- L = loss
- $\frac{\partial L}{\partial W}$ = gradient
- η = learning rate

1.1 Batch Gradient Descent (BGD)

Batch Gradient Descent computes the gradient using the entire training dataset before performing a single weight update. It is precise and produces a smooth, deterministic descent toward the minimum, but it is computationally expensive and memory-intensive for large datasets, and it updates weights only once per epoch.

Then:

$$\nabla L(W) = \frac{1}{N} \sum_{i=1}^N \nabla L_i(W)$$

and:

$$W \leftarrow W - \eta \nabla L(W)$$

Where N is the total no of datapoints in our datasets.

Advantages

- Stable gradient
- Smooth convergence
- Useful for relatively small datasets

Disadvantages

- Computationally expensive
- Requires more memory
- Slow updates for very large datasets

1.2 Mini-Batch Gradient Descent

Mini-Batch Gradient Descent splits the training set into small batches (commonly 32, 64, 128 samples) and performs a weight update after processing each batch. It strikes a balance between the stability of Batch GD and the speed and regularizing noise of pure SGD, and it is the default choice in almost all modern deep learning frameworks because it also maps efficiently onto GPU parallelism.

The update is:

$$W \leftarrow W - \eta \frac{1}{B} \sum_{i=1}^B \nabla L_i(W)$$

where B is batch size.

1.3 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent updates the weights after every single training example. This makes each update extremely fast and injects noise into the descent path, which can help the optimizer escape shallow local minima and saddle points — but the noisy path also means the loss oscillates rather than decreasing smoothly, and convergence to the exact minimum requires a decaying learning rate.

2. Advanced Optimizers

In Ordinary Gradient Descent every update is based **only on the current gradient**.

It has no memory of what happened in previous iterations.

Momentum-based optimizers fix this by giving the optimizer a form of physical inertia — remembering the direction of past updates so it can accelerate through consistent gradient directions and dampen oscillations.

2.1 Momentum

Momentum accumulates an exponentially decaying moving average of past gradients (called the velocity, v) and uses that velocity — rather than the raw gradient — to update the weights. It behaves like a ball rolling downhill that gathers speed along consistent slopes and is less affected by small bumps or noisy gradients.

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w_t)$$
$$w_{t+1} = w_t - \eta \cdot v_t$$

β (typically 0.9) controls how much of the past velocity is retained. Higher β means more inertia and smoother, faster convergence along consistent directions, at the cost of possible overshoot near the minimum.

2.2 Nesterov Accelerated Gradient (NAG)

NAG improves on plain Momentum with a 'look-ahead' correction. Instead of computing the gradient at the current position, it first takes a step in the direction of the accumulated velocity, then computes the gradient at that look-ahead point. This gives the optimizer a preview of where it is heading, letting it correct course earlier and reduce overshoot compared to standard Momentum.

$$w_look = w_t - \eta \cdot \beta \cdot v_{t-1}$$
$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w_look)$$
$$w_{t+1} = w_t - \eta \cdot v_t$$

Optimizer	Key Idea	Main Benefit	Main Risk
Momentum	EMA of past gradients as velocity	Faster, smoother convergence	Can overshoot the minimum
NAG	Gradient evaluated at look-ahead point	Corrects course earlier than Momentum	Slightly more compute per step

3. Adaptive Learning Methods

Momentum and NAG still use a single, global learning rate for every weight. Adaptive methods go a step further: they maintain a per-parameter learning rate that automatically shrinks or grows based on how that specific parameter's gradients have behaved historically. This is especially valuable when features have very different scales or frequencies (e.g. sparse features in NLP).

3.1 AdaGrad (Adaptive Gradient Algorithm)

AdaGrad divides the learning rate for each parameter by the square root of the sum of all its squared past gradients. Parameters that have received large or frequent gradients get their effective learning rate reduced, while parameters with small or infrequent gradients keep a relatively larger learning rate. This makes AdaGrad well suited to sparse data, but the continually accumulating denominator means the effective learning rate keeps shrinking and can become vanishingly small, causing learning to stall in long training runs.

$$G_t = G_{t-1} + (\nabla L(w_t))^2$$
$$w_{t+1} = w_t - (\eta / \sqrt{G_t + \epsilon}) \cdot \nabla L(w_t)$$

3.2 RMSProp (Root Mean Square Propagation)

RMSProp fixes AdaGrad's vanishing learning-rate problem by replacing the ever-growing sum of squared gradients with an exponentially decaying moving average. This means old gradients are gradually 'forgotten', so the effective learning rate does not shrink to zero, making RMSProp much better suited to non-stationary objectives such as deep network training.

$$S_t = \beta \cdot S_{t-1} + (1 - \beta) \cdot (\nabla L(w_t))^2$$
$$w_{t+1} = w_t - (\eta / \sqrt{S_t + \epsilon}) \cdot \nabla L(w_t)$$

3.3 Adam (Adaptive Moment Estimation)

Adam combines the ideas of Momentum and RMSProp: it keeps an exponentially decaying average of past gradients (the first moment, like Momentum's velocity) and an exponentially decaying average of past squared gradients (the second moment, like RMSProp's scaling term). Both moments are bias-corrected to account for their initialization at zero, which matters most during the earliest steps of training. Adam is the most widely used optimizer in modern deep learning because it converges quickly and needs little manual learning-rate tuning.

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla L(w_t)$$
$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (\nabla L(w_t))^2$$
$$\hat{m}_t = m_t / (1 - \beta_1^t)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$w_{t+1} = w_t - (\eta / (\sqrt{\hat{v}_t} + \varepsilon)) \cdot \hat{m}_t$$

Optimizer	Adapts LR per-parameter?	Uses momentum?	Typical use
SGD	No	No	Simple baselines, well-tuned schedules
Momentum / NAG	No	Yes	Convex-ish, well-conditioned losses
AdaGrad	Yes (monotonic decay)	No	Sparse features (e.g. NLP embeddings)
RMSProp	Yes (EMA-based)	No	RNNs, non-stationary objectives
Adam	Yes (EMA-based)	Yes	Default choice for most deep nets

4. Gradient Challenges: Vanishing & Exploding Gradients

Backpropagation computes gradients for early layers by chaining derivatives through every layer that comes after it (the chain rule). In deep networks this chain can contain dozens or hundreds of multiplicative terms, and this is where two opposite but related problems arise.

4.1 Vanishing Gradients

When each layer's local derivative is less than 1 (which happens with saturating activations like sigmoid or tanh, whose derivatives max out at 0.25 and 1.0 respectively but are usually much smaller), multiplying many such small numbers together across layers makes the gradient shrink exponentially as it propagates backward. Early layers then receive a gradient close to zero and effectively stop learning, even though later layers continue to update normally.

$$\partial L / \partial w_1 = \partial L / \partial a_n \cdot \partial a_n / \partial a_{n-1} \cdot \dots \cdot \partial a_2 / \partial a_1 \cdot \partial a_1 / \partial w_1$$

If every factor in that product is small, the overall gradient for w_1 vanishes.

Common Fixes

- Use ReLU / Leaky ReLU / GELU instead of sigmoid or tanh in hidden layers — their derivative is 1 for positive inputs, so it does not shrink the gradient.
- Use residual (skip) connections so gradients have a direct path back, bypassing long multiplicative chains.
- Apply Batch Normalization or Layer Normalization to keep activations in a well-behaved range (see Section 5).
- Use careful weight initialization (Xavier/Glorot for tanh/sigmoid, He initialization for ReLU).
- Use architectures designed to combat this in sequence models: LSTM/GRU gating mechanisms.

4.2 Exploding Gradients

The opposite problem occurs when the local derivatives (or the weight magnitudes) are consistently greater than 1. The chain-rule product then grows exponentially as it propagates backward, producing huge gradient values. This causes wildly oscillating or NaN/Inf loss values and destabilizes training entirely — most commonly seen in deep and recurrent networks with poorly scaled weights.

Common Fixes

- Gradient Clipping — rescale the gradient vector if its norm exceeds a threshold (shown below).
- Careful weight initialization to keep the initial signal variance stable across layers.
- Lower the learning rate, or use a warm-up schedule at the start of training.
- Use Batch/Layer Normalization to control the scale of activations layer to layer.

Problem	Root Cause	Symptom	Primary Fixes
Vanishing gradient	Repeated multiplication by derivatives < 1	Early layers stop learning; loss plateaus	ReLU family, skip connections, BatchNorm, good init
Exploding gradient	Repeated multiplication by values > 1	Loss oscillates or becomes NaN/Inf	Gradient clipping, lower LR, good init, normalization

5. Normalization Techniques

Normalization layers standardize the distribution of activations flowing through the network. By keeping activations centered around zero with a controlled variance, they combat vanishing/exploding gradients, allow the use of larger learning rates, and act as a mild regularizer — all of which speed up and stabilize training.

5.1 Batch Normalization (BatchNorm)

Batch Normalization normalizes each activation across the mini-batch dimension: for a given layer, it computes the mean and variance of each feature over the current mini-batch, normalizes using those statistics, and then applies a learnable scale (γ) and shift (β) so the network can undo the normalization if that is actually optimal. At inference time, running averages of the mean and variance (accumulated during training) are used instead of batch statistics.

$$\begin{aligned}\mu_B &= (1/m)\sum x_i \\ \sigma_B^2 &= (1/m)\sum (x_i - \mu_B)^2 \\ \hat{x}_i &= (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)} \\ y_i &= \gamma \cdot \hat{x}_i + \beta\end{aligned}$$

5.2 Layer Normalization (LayerNorm)

Layer Normalization normalizes across the feature dimension for each individual sample, instead of across the batch dimension. This makes it independent of batch size — it works identically whether the batch size is 1 or 1000 — which is why it is the default choice in Transformers and RNNs, where batch statistics can be unreliable (e.g. variable sequence lengths) or where batch size may be very small.

$$\mu_i = (1/d)\sum_j x_{ij} \quad \sigma_i^2 = (1/d)\sum_j (x_{ij} - \mu_i)^2 \quad \hat{x}_{ij} = (x_{ij} - \mu_i) / \sqrt{(\sigma_i^2 + \epsilon)} \quad y_{ij} = \gamma_j \cdot \hat{x}_{ij} + \beta_j$$

Aspect	Batch Normalization	Layer Normalization
Normalizes across	The batch dimension (per feature)	The feature dimension (per sample)
Depends on batch size?	Yes — unstable for very small batches	No — works even with batch size 1
Needs running stats at inference?	Yes	No
Typical use case	CNNs, standard feedforward nets	Transformers, RNNs, small-batch training

6. Regularization & Hyperparameter Tuning

A network that fits the training data almost perfectly but performs poorly on unseen data has overfit — it has memorized noise and idiosyncrasies of the training set rather than learning generalizable patterns. Regularization techniques constrain the model to prevent this, while tuning techniques search for the hyperparameter configuration (learning rate, regularization strength, architecture size, etc.) that generalizes best.

6.1 L1 and L2 Regularization

Both techniques add a penalty term to the loss function based on the magnitude of the weights, discouraging the model from relying too heavily on any single weight.

L2 Regularization (Ridge / Weight Decay)

L2 regularization adds the sum of squared weights to the loss. It shrinks all weights smoothly toward zero without forcing any of them to become exactly zero, which tends to produce diffuse, well-conditioned weight vectors.

$$L_{total} = L_{original} + (\lambda/2) \cdot \sum w^2 \quad \partial L_{total} / \partial w = \partial L_{original} / \partial w + \lambda w$$

L1 Regularization (Lasso)

L1 regularization adds the sum of absolute weight values to the loss. Its gradient is a constant ($\pm\lambda$) regardless of weight magnitude, which pushes many weights all the way to exactly zero — effectively performing automatic feature selection and producing sparse models.

$$L_{total} = L_{original} + \lambda \cdot \sum |w| \quad \partial L_{total} / \partial w = \partial L_{original} / \partial w + \lambda \cdot \text{sign}(w)$$

6.2 Dropout

Dropout randomly deactivates (zeroes out) a fraction p of neurons during each training forward pass, forcing the network to not rely on any single neuron or narrow co-adapted group of neurons. This is equivalent to training an ensemble of many thinned sub-networks that share weights. At test time, dropout is switched off and, to keep the expected output magnitude consistent, the remaining activations are scaled — this is the common 'inverted dropout' implementation.

$$\text{mask} \sim \text{Bernoulli}(1-p) \quad a_{\text{dropped}} = (a \odot \text{mask}) / (1-p) \quad (\text{training only})$$

6.3 Early Stopping

Early stopping monitors performance on a held-out validation set during training and halts training once validation loss stops improving for a set number of epochs (the 'patience'), even if training loss is still decreasing. This directly prevents the model from continuing to fit noise in the training data after it has already learned the generalizable signal.

6.4 Data Augmentation

Data augmentation artificially expands the effective size and diversity of the training set by applying realistic, label-preserving transformations to existing samples (e.g. flips, rotations, crops, noise for images; synonym replacement for text). The model sees more variety without needing more collected data, which reduces overfitting because it can no longer memorize exact training samples.

Practical Implementation — Basic Image Augmentation

```
import numpy as np

def random_flip(image):
    if np.random.rand() > 0.5:
        return np.fliplr(image)
    return image

def random_rotation(image, max_angle=15):
    from scipy.ndimage import rotate
    angle = np.random.uniform(-max_angle, max_angle)
    return rotate(image, angle, reshape=False, mode='nearest')

def add_gaussian_noise(image, std=0.05):
    noise = np.random.normal(0, std, image.shape)
    return np.clip(image + noise, 0, 1)

def random_crop_and_pad(image, crop_size=24, pad=4):
    padded = np.pad(image, pad, mode='edge')
    h, w = image.shape
    top = np.random.randint(0, padded.shape[0] - crop_size)
    left = np.random.randint(0, padded.shape[1] - crop_size)
    return padded[top:top + crop_size, left:left + crop_size]

def augment(image):
    image = random_flip(image)
    image = random_rotation(image)
    image = add_gaussian_noise(image)
    return image

# Example on a synthetic 28x28 "image"
np.random.seed(3)
```

```
img = np.random.rand(28, 28)
augmented = augment(img)
print("Original shape:", img.shape, " Augmented shape:", augmented.shape)
```

6.5 Hyperparameter Tuning: Grid Search, Random Search & Bayesian Optimization

Beyond regularizing a fixed architecture, generalization also depends heavily on hyperparameters such as learning rate, regularization strength λ , batch size, and network width/depth. These three strategies search the hyperparameter space to find a configuration that performs well on validation data.

Grid Search

Grid Search exhaustively evaluates every combination of hyperparameter values from a predefined set. It is simple and guarantees the best configuration within the grid, but the number of combinations grows exponentially with the number of hyperparameters, making it expensive for anything beyond 2–3 hyperparameters.

```
from itertools import product

def grid_search(param_grid, eval_fn):
    """
    param_grid: dict of {param_name: [values]}
    eval_fn: function(params_dict) -> validation_score (higher is better)
    """
    keys = list(param_grid.keys())
    best_score = -np.inf
    best_params = None

    for combo in product(*param_grid.values()):
        params = dict(zip(keys, combo))
        score = eval_fn(params)
        if score > best_score:
            best_score = score
            best_params = params

    return best_params, best_score

# Dummy evaluation function standing in for "train model & return val accuracy"
def dummy_eval(params):
    lr, lam = params["lr"], params["lambda"]
    # Pretend best region is around lr=0.01, lambda=0.01
    return -((np.log10(lr) + 2) ** 2 + (np.log10(lam) + 2) ** 2)

grid = {
    "lr": [0.001, 0.01, 0.1],
    "lambda": [0.0001, 0.001, 0.01],
}

best_params, best_score = grid_search(grid, dummy_eval)
print("Best params (Grid Search):", best_params, "score:", round(best_score, 4))
```

Random Search

Random Search samples hyperparameter combinations randomly from specified distributions rather than testing every combination. Perhaps counterintuitively, this is usually more efficient than Grid Search with the same compute budget, because it explores more distinct values along each individual hyperparameter dimension instead of wasting evaluations on a rigid grid.

```
def random_search(param_distributions, eval_fn, n_iter=20, seed=0):
    """
    param_distributions: dict of {param_name: sampling_function}
    """
    rng = np.random.default_rng(seed)
    best_score = -np.inf
    best_params = None

    for _ in range(n_iter):
        params = {k: dist(rng) for k, dist in param_distributions.items()}
        score = eval_fn(params)
        if score > best_score:
            best_score = score
            best_params = params

    return best_params, best_score

param_dist = {
    "lr": lambda rng: 10 ** rng.uniform(-4, 0),          # log-uniform between 1e-4 and 1
    "lambda": lambda rng: 10 ** rng.uniform(-5, -1),
}

best_params, best_score = random_search(param_dist, dummy_eval, n_iter=30)
print("Best params (Random Search):", {k: round(v, 5) for k, v in best_params.items()},
      "score:", round(best_score, 4))
```

Bayesian Optimization

Bayesian Optimization treats hyperparameter search as an optimization problem over an unknown, expensive-to-evaluate function. It builds a probabilistic surrogate model (commonly a Gaussian Process) of how validation score depends on hyperparameters, and uses that surrogate — plus an acquisition function balancing exploration and exploitation — to intelligently choose the next, most promising configuration to try. This makes far better use of a limited evaluation budget than Grid or Random Search, at the cost of extra algorithmic complexity.

```
# Minimal Bayesian Optimization using a Gaussian Process surrogate (scikit-learn)
# pip install scikit-learn scipy

import numpy as np
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern
from scipy.stats import norm
```

```

def expected_improvement(X, gp, y_best, xi=0.01):
    mu, sigma = gp.predict(X, return_std=True)
    sigma = sigma.reshape(-1, 1)
    mu = mu.reshape(-1, 1)
    with np.errstate(divide='warn'):
        imp = mu - y_best - xi
        Z = imp / sigma
        ei = imp * norm.cdf(Z) + sigma * norm.pdf(Z)
        ei[sigma == 0.0] = 0.0
    return ei.ravel()

def bayesian_optimization(eval_fn, bounds, n_init=5, n_iter=15, seed=0):
    rng = np.random.default_rng(seed)
    dim = bounds.shape[0]

    # Random initial samples to seed the surrogate model
    X_sample = rng.uniform(bounds[:, 0], bounds[:, 1], size=(n_init, dim))
    y_sample = np.array([eval_fn(x) for x in X_sample]).reshape(-1, 1)

    kernel = Matern(nu=2.5)
    gp = GaussianProcessRegressor(kernel=kernel, alpha=1e-6, normalize_y=True)

    for i in range(n_iter):
        gp.fit(X_sample, y_sample)

        # Search for the point that maximizes Expected Improvement
        candidates = rng.uniform(bounds[:, 0], bounds[:, 1], size=(1000, dim))
        ei = expected_improvement(candidates, gp, y_sample.max())
        x_next = candidates[np.argmax(ei)].reshape(1, -1)

        y_next = np.array([[eval_fn(x_next[0])]])
        X_sample = np.vstack([X_sample, x_next])
        y_sample = np.vstack([y_sample, y_next])

    best_idx = np.argmax(y_sample)
    return X_sample[best_idx], y_sample[best_idx][0]

# Objective: log10(lr), log10(lambda) -> validation score (dummy peak at (-2,-2))
def objective(x):
    log_lr, log_lam = x
    return -((log_lr + 2) ** 2 + (log_lam + 2) ** 2)

bounds = np.array([[-4, 0], [-5, -1]]) # search range for log10(lr), log10(lambda)
best_x, best_y = bayesian_optimization(objective, bounds, n_init=5, n_iter=15)
print(f"Best log10(lr)={best_x[0]:.3f}, log10(lambda)={best_x[1]:.3f}, score={best_y:.4f}")

```